

Problem Set 4 Code

● Graded

11 Minutes Late

Student

Jinku Jung

Total Points

20 / 20 pts

Autograder Score

20.0 / 20.0

Passed Tests

Part C - Replace Evens (2/2)

Part F - Move Evens to Front (2/2)

Part A - Swap First & Last (2/2)

Part B - Shift Right (2/2)

Part J - Check for Duplicates (2/2)

Part D - Replace with Larger (2/2)

Part G - Find Second Largest (2/2)

Part E - Remove Middle Element (2/2)

Part I - Check for Adjacent Duplicates (2/2)

Part H - Check if Sorted (2/2)

Question 2

Autograder Adjustment

0 / 0 pts

✓ + 0 pts No adjustment

+ 0 pts Adjustment needed

Autograder Results

Part C - Replace Evens (2/2)

Part F - Move Evens to Front (2/2)

Part A - Swap First & Last (2/2)

Part B - Shift Right (2/2)

Part J - Check for Duplicates (2/2)

Part D - Replace with Larger (2/2)

Part G - Find Second Largest (2/2)

Part E - Remove Middle Element (2/2)

Part I - Check for Adjacent Duplicates (2/2)

Part H - Check if Sorted (2/2)

Submitted Files

```
1  /*
2   * ArrayMethods - Manipulate Array Contents
3   *
4   * Problem Set #4 Code for E 7.11: (a) through (j)
5   *
6   * COMS W1004
7   *
8   * Name: Jinku Jung
9   * UNI: dj2598
10  * Date: 03/22/2021
11  *
12  */
13
14  // Cut & pasted from VS Code (Windows 10 Pro) to test my
15  // final submission. This file was downloaded
16  // from Gradescope AFTER my Problem Set #4 Java code
17  // was uploaded to Gradescope to ensure it compiles!
18  //
19  //      --Jinku 10:22 PM EST on 3/22/2021
20
21  import java.util.Arrays;
22  import java.util.ArrayList;
23
24  public class ArrayMethods {
25
26      // private instance variables
27      private int[] values;
28      // this is for resetting purposes, otherwise not needed
29      private int[] originalValues;
30
31      //constructor
32      public ArrayMethods(int[] initialValues) {
33          values = initialValues;
34          originalValues = Arrays.copyOf(values, values.length);
35      }
36
37      // pretty printing for testing purposes
38      public void printArray() {
39          for (int i : values) {
40              System.out.print(i + " ");
41          }
42          System.out.println("");
43      }
44
45      // resetting back to original values for testing purposes
46      public void resetValues() {
47          values = Arrays.copyOf(originalValues, originalValues.length);
48      }
49  }
```

```
50 // a. swap the first and last elements
51 public void swapFirstAndLast() {
52
53     int last = values.length - 1;
54     int temp = values[0];
55
56     values[0] = values[last];
57     values[last] = temp;
58
59 }
60
61 // b. shift all element to right and wraparound
62 public void shiftRight() {
63
64     int last = values.length - 1;
65     int temp = values[last];
66
67     for (int i = last; i > 0; i--)
68     {
69         values[i] = values[i - 1];
70     }
71
72     values[0] = temp;
73
74 }
75
76 // c. replace even elements with zero
77 public void replaceEven() {
78
79     for (int i = 0; i < values.length; i++)
80     {
81         if (values[i] % 2 == 0) // Verifies if the number is even.
82         {
83             values[i] = 0;
84         }
85     }
86 }
87
88 // d. replace middle elements with larger of two neighbors
89 public void replaceMiddle() {
90
91     int[] temp = new int[values.length];
92
93     for(int j = 0; j < values.length; j++) {
94         temp[j] = values[j];
95     }
96
97
98     for (int i = 1; i < (values.length - 1); i++)
99     {
100         if (values[i - 1] < values[i + 1])
101         {
```

```

102         temp[i] = values[i+1];
103     }
104     if (values[i - 1] > values[i + 1])
105     {
106         temp[i] = values[i - 1];
107     }
108     if (values[i-1] == values[i+1])
109     {
110         temp[i] = values[i+1];
111     }
112 }
113 values = temp;
114 }
115
116
117 // e. Remove middle one or two elements
118 public void removeMiddle() {
119
120     int size = values.length;
121
122     if (size % 2 == 0) // Size is even...
123     {
124         // Find the first element of the "middle."
125         int firstMiddle = size / 2 - 1;
126
127         // Remove the middle two elements.
128         for (int i = firstMiddle; i < size - 2; i++)
129         {
130             values[i] = values[i + 2];
131         }
132
133         int[] temp = new int[size - 2];
134
135         for(int j = 0; j < size - 2; j++) {
136             temp[j] = values[j];
137         }
138
139         values = temp;
140     }
141
142     else // Size HAS TO BE odd!
143
144     {
145         // Find the first element of the "middle."
146         int firstMiddle = size / 2;
147
148         // Remove the middle element.
149         for (int i = firstMiddle; i < size - 1; i++)
150         {
151             values[i] = values[i + 1];
152         }
153

```

```

154     int[] temp = new int[size - 1];
155
156     for(int j = 0; j < size - 1; j++) {
157         temp[j] = values[j];
158     }
159
160     values = temp;
161
162 }
163 }
164
165 // f. Move even elements to the front
166 public void moveEven() {
167
168     int lastEven = 0;
169     int temp;
170
171     for (int i = 0; i < values.length; i++)
172     {
173         if (values[i] % 2 == 0) // Checks for an even value.
174         {
175             temp = values[i]; // Saves the said even value to a
176                             // dummy variable.
177
178             // Now, move an even-valued element rightward toward the
179             // front of the array, while preserving the order of
180             // the other odd-valued elements in the rest of the
181             // array.
182             for (int j = i; j > lastEven; j--)
183             {
184                 values[j] = values[j - 1];
185             }
186
187             values[lastEven] = temp;
188             lastEven++;
189         }
190     }
191 }
192
193 // g. Return the value of the second-largest element
194 public int secondLargest() {
195     // 1.) Find the true maximum value in the given array.
196     int max = values[0];
197
198     for (int i = 1; i < values.length; i++)
199     {
200         if (values[i] > max)
201         {
202             max = values[i];
203         }
204     }
205

```

```

206 // 2.) Find the max again, now ignoring the true maximum
207 // value of the given array.
208
209 int oldMax = max;
210 max = values[0];
211
212 for (int i = 1; i < values.length; i++)
213 {
214     if (values[i] != oldMax)
215     {
216         if (values[i] > max)
217         {
218             max = values[i];
219         }
220     }
221 }
222
223 return max;
224 }
225
226 // h. Check if sorted in increasing order
227 public boolean sortedIncreasing() {
228     // 1.) Assume the array elements are in increasing order.
229     boolean areOrdered = true;
230
231     // 2.) Step through array & actually verify if its elements
232     // are sequentially ordered--according to increasing value.
233     for (int i = 0; i < values.length - 1; i++)
234     {
235         if (values[i] > values[i + 1])
236         {
237             areOrdered = false;
238         }
239     }
240
241     return areOrdered;
242 }
243
244 // i. Check if contains two adjacent duplicate elements
245 public boolean adjacentDups() {
246     // 1.) Assume no adjacent dupes exist.
247     boolean areAdjDups = false;
248
249     // 2.) Step through array to verify if
250     // any adjacent duplicate values
251     // really exist.
252     for (int i = 0; i < values.length - 1; i++)
253     {
254         if (values[i] == values[i + 1])
255         {
256             areAdjDups = true;
257         }

```

```
258     }
259
260     return areAdjDups;
261 }
262
263 // j. Check if contains any duplicate elements
264 public boolean containsDups() {
265     // 1.) Assume no dups exist.
266     boolean hasDups = false;
267
268     // 2.) Step through the array to verify if
269     //     any dups really exist.
270
271     for (int i = 0; i < values.length; i++)
272     {
273         for (int j = i + 1; j < values.length; j++)
274         {
275             if (values[i] == values[j])
276             {
277                 hasDups = true;
278             }
279         }
280     }
281
282     return hasDups;
283 }
284 }
285
```

```
1  /**
2   * ArrayMethodsTester - Tester for Array Methods
3   *
4   * COMS W1004
5   *
6   *
7   *
8   * @author Kimberly Hou (kjh2146)
9   */
10
11 public class ArrayMethodsTester {
12     public static void main(String[] args){
13         int[] x = { -1, 2, 4, 2, 7, 8, 9 };
14
15         ArrayMethods am = new ArrayMethods(x);
16
17         // original array
18         am.printArray();
19
20         // test a
21         am.swapFirstAndLast();
22         System.out.print("Swapping: ");
23         am.printArray();
24         am.resetValues();
25
26         // test b
27         am.shiftRight();
28         System.out.print("Shifting right: ");
29         am.printArray();
30         am.resetValues();
31
32         // test c
33         am.replaceEven();
34         System.out.print("Replacing even elements: ");
35         am.printArray();
36         am.resetValues();
37
38         // test d
39         am.replaceMiddle();
40         System.out.print("Replacing middle elements: ");
41         am.printArray();
42         am.resetValues();
43
44         // test e
45         am.removeMiddle();
46         System.out.print("Removing middle element(s): ");
47         am.printArray();
48         am.resetValues();
49     }
```

```
50 // test f
51 am.moveEven();
52 System.out.print("Moving even elements: ");
53 am.printArray();
54 am.resetValues();
55
56 // test g
57 System.out.print("Second largest element: ");
58 System.out.println(am.secondLargest());
59
60 // test h
61 System.out.print("Is sorted in increasing order: ");
62 System.out.println(am.sortedIncreasing());
63
64 // test i
65 System.out.print("Has adjacent duplicates: ");
66 System.out.println(am.adjacentDups());
67
68 // test j
69 System.out.print("Contains duplicates: ");
70 System.out.println(am.containsDups());
71
72
73 }
74
75 }
76
```
